

Windows Notification System - Comprehensive Technical Reference v2.0

Document Version: 2.0

Last Updated: 2026-09-19

For: CreateEmailEventAndTeamsChat.Prototype.ps1 Notifications Feature

Executive Summary

This document provides a comprehensive technical reference for implementing Windows toast notifications using the Windows Runtime (WinRT) API in PowerShell applications.

Key Findings:

- **Notification Architecture:** Windows 10/11 use the WinRT Toast Notification API for modern desktop alerts. Notifications appear in the lower-right corner for 5-25 seconds (configurable), then persist in Action Center for up to 7 days. The system supports rich content including images, sounds, progress bars, and interactive buttons.
- **Multiple Notifications:** Windows queues notifications intelligently, displaying a maximum of 3 toasts simultaneously on screen. All notifications are stored in Action Center with a limit of 20 per application. Tag and Group properties enable notification replacement and threading for better organization.
- **Persistence and Loss:** Notifications persist through reboots and are stored in Action Center for 5 days by default (max 7 days). However, Focus Assist can suppress notifications completely without storing them. The system has no built-in authentication—any application running as the current user can send notifications under any App ID.
- **Cross-Device Sync:** Optional but unreliable. If users enable notification sync via Microsoft Account, toasts may appear on multiple devices. However, this feature uses best-effort delivery with 15-30 minute delays and is not suitable for critical notifications. Local file paths and custom sounds do not sync.
- **Security Model:** User-level only with no App ID authentication. Applications are trusted to display accurate content. Notification content is logged in Action Center's SQLite database and may sync to Microsoft servers if cross-device sync is enabled. Do not include PII or sensitive data in notification text.
- **PowerShell Implementation:** Full support via COM-style WinRT type loading. The implementation pattern requires loading three WinRT types, building XML toast templates, and calling the ToastNotificationManager. The codebase already has a working implementation in TrackSessions.Simulator CR1.ps1 that can be adapted.

Quick Reference:

- Basic notification: 5 lines of PowerShell (load types, create XML, show)
- Supported sounds: Default, Mail, Reminder, SMS, IM, Silent
- Duration options: Short (5s), Default (7s), Long (25s)

- Max 20 notifications per app in Action Center
 - Tag enables replacement, Group enables threading
 - Focus Assist can silently suppress all notifications
-

Table of Contents

1. [Overview](#)
2. [How Windows Notifications Work](#)
 - [2.1 Architecture](#)
 - [2.2 Process Flow](#)
3. [Toast Notification Framework](#)
 - [3.1 Basic Template Types](#)
 - [3.2 Modern Adaptive Templates](#)
4. [Multiple Notifications from Same Application](#)
 - [4.1 How Windows Handles Multiple Toasts](#)
 - [4.2 Sending Multiple Notifications](#)
 - [4.3 Notification Throttling Best Practice](#)
5. [Grouping and Threading](#)
 - [5.1 Tag and Group System](#)
 - [5.2 Grouped Notifications Example](#)
 - [5.3 Header-Based Grouping](#)
 - [5.4 Replacing Notifications](#)
6. [Silencing Notifications](#)
 - [6.1 User-Level Controls](#)
 - [6.2 Programmatic Silencing](#)
7. [Notification Persistence and Loss Prevention](#)
 - [7.1 Do Notifications Get Lost?](#)
 - [7.2 When Notifications Persist](#)
 - [7.3 When Notifications Are Lost](#)
 - [7.4 Ensuring Notifications Don't Get Lost](#)
8. [Customizing the Toast UI](#)
 - [8.1 Toast Display Duration](#)
 - [8.2 Adding Images](#)
 - [8.3 Interactive Buttons](#)
 - [8.4 Text Input Fields](#)
 - [8.5 Progress Bar](#)
 - [8.6 Sounds](#)
9. [Security Model](#)
 - [9.1 Application ID Registration](#)
 - [9.2 Notification Content Security](#)
 - [9.3 Privacy Considerations](#)
 - [9.4 Code Execution Security](#)
 - [9.5 Sandboxing](#)
10. [Multi-Machine Considerations](#)
 - [10.1 Cross-Device Notifications](#)
 - [10.2 Best Practices for Multi-Machine Scenarios](#)

- 10.3 [Notification Action Center Sync Behavior](#)
 - 11. [PowerShell Implementation Guide](#)
 - 11.1 [Minimal Working Example](#)
 - 11.2 [Production-Ready Implementation](#)
 - 11.3 [Helper Functions](#)
 - 12. [Advanced Features](#)
 - 12.1 [Scheduled Notifications](#)
 - 12.2 [Notification Update \(Progress Bar\)](#)
 - 12.3 [Notification with Buttons \(Advanced\)](#)
 - 13. [Summary and Feature Matrix](#)
 - 14. [Glossary](#)
-

1. Overview

[↑ Top](#)

Windows 10 and Windows 11 use the **Windows Runtime (WinRT) Toast Notification API** for modern app notifications. This system replaced the older balloon tip notifications and provides:

- Rich, interactive notifications with buttons, images, progress bars
- Notification grouping and threading
- Persistent notification history in Action Center
- User-controlled Do Not Disturb (Focus Assist)
- Per-app notification settings and permissions
- Cross-device notification sync (optional)

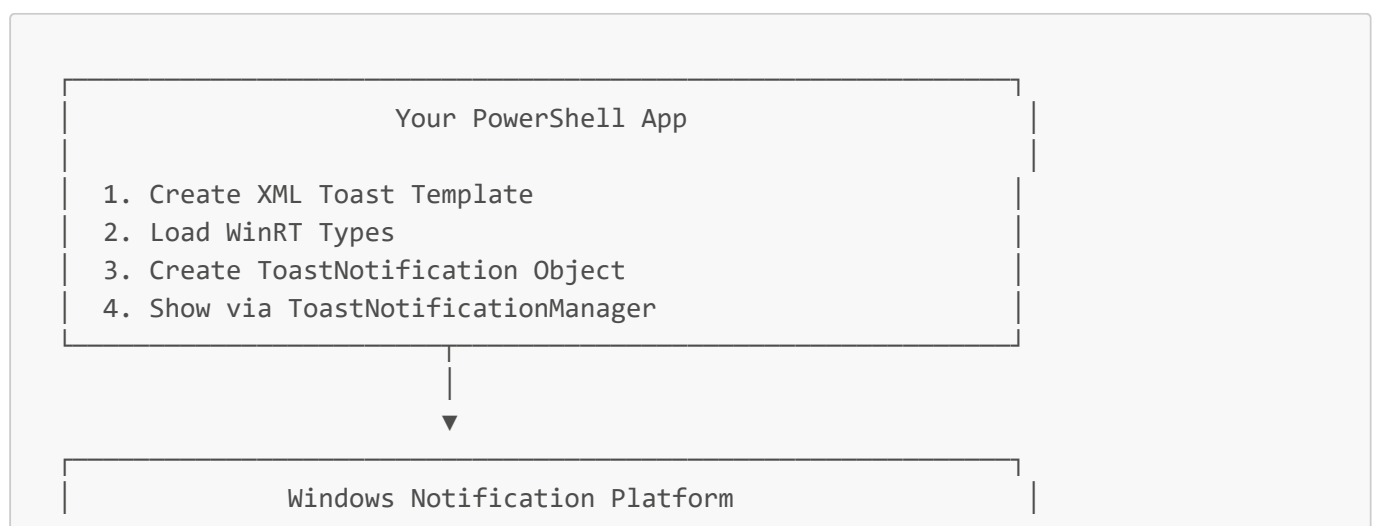
Key Namespace: `Windows.UI.Notifications`

2. How Windows Notifications Work

[↑ Top](#)

2.1 Architecture

[↑ Top](#)



- Validates XML Schema
- Checks App ID Registration
- Applies User Preferences (Focus Assist, Per-App)
- Queues Notification
- Renders Toast in Top-Right Corner
- Archives to Action Center (Win+A)



Action Center History

- Stores up to 20 notifications per app
- User can clear individually or in bulk
- Syncs across devices (if enabled)
- Persists through reboots

2.2 Process Flow

[↑ Top](#)

1. **App ID Registration:** When you call `CreateToastNotifier($APP_ID)`, Windows creates or uses an existing registration for that App ID in the registry.
2. **XML Template:** You define the notification content in XML format following the Toast Content Schema.
3. **WinRT Types:** PowerShell loads the Windows Runtime types from `Windows.UI.Notifications.dll` which ships with Windows 10/11.
4. **Show():** Calling `.Show()` submits the notification to the Windows notification broker service.
5. **Rendering:** Windows displays the toast in the lower-right corner for ~5 seconds (default), then moves it to Action Center.

3. Toast Notification Framework

[↑ Top](#)

3.1 Basic Template Types

[↑ Top](#)

Windows provides several built-in toast templates:

Template	Description	Fields
<code>ToastText01</code>	Single line of text	Title only
<code>ToastText02</code>	Two lines of text	Title + Message

Template	Description	Fields
ToastText03	Two lines with wrapped text	Title + Long Message
ToastText04	Three lines	Title + Message + Secondary Message
ToastImageAndText01	Image + one line	Image + Title
ToastImageAndText02	Image + two lines	Image + Title + Message
ToastImageAndText03	Image + wrapped text	Image + Title + Long Message
ToastImageAndText04	Image + three lines	Image + Title + Message + Secondary

3.2 Modern Adaptive Templates

[↑ Top](#)

Windows 10+ supports **adaptive toast templates** with richer layouts:

```
<toast>
  <visual>
    <binding template="ToastGeneric">
      <text>Title Text</text>
      <text>Body Text Line 1</text>
      <text>Body Text Line 2</text>
      <image placement="appLogoOverride" src="C:\path\to\icon.png"/>
      <image placement="hero" src="C:\path\to\banner.png"/>
    </binding>
  </visual>
  <actions>
    <action content="Reply" arguments="reply" />
    <action content="Dismiss" arguments="dismiss" />
  </actions>
  <audio src="ms-winsoundevent:Notification.Default" loop="false" />
</toast>
```

Key Elements:

- `<visual>`: Display content (text, images)
- `<actions>`: Interactive buttons (up to 5)
- `<audio>`: Sound file or Windows system sound
- `<header>`: For grouping notifications

4. Multiple Notifications from Same Application

[↑ Top](#)

4.1 How Windows Handles Multiple Toasts

[↑ Top](#)

When you show multiple notifications from the same App ID:

1. **Queuing:** Windows queues notifications if more than 3-4 are shown rapidly
2. **Display Limit:** Maximum 3 toasts visible simultaneously on screen
3. **Action Center:** All notifications stored in history (up to 20 per app)
4. **Replacement:** New notifications push older ones to Action Center faster

4.2 Sending Multiple Notifications

[↑ Top](#)

```
function Show-WindowsNotification {
    param(
        [string]$Title,
        [string]$Message,
        [string]$Tag = $null,      # Unique identifier for replacement
        [string]$Group = $null    # Group identifier for threading
    )

    [Windows.UI.Notifications.ToastNotificationManager, Windows.UI.Notifications,
    ContentType = WindowsRuntime] | Out-Null
    [Windows.UI.Notifications.ToastNotification, Windows.UI.Notifications,
    ContentType = WindowsRuntime] | Out-Null
    [Windows.Data.Xml.Dom.XmlDocument, Windows.Data.Xml.Dom.XmlDocument,
    ContentType = WindowsRuntime] | Out-Null

    $APP_ID = "CreateEmailEventAndTeamsChat"
    $template = @"
<toast>
    <visual>
        <binding template="ToastText02">
            <text id="1">$Title</text>
            <text id="2">$Message</text>
        </binding>
    </visual>
</toast>
"@

    $xml = New-Object Windows.Data.Xml.Dom.XmlDocument
    $xml.LoadXml($template)
    $toast = New-Object Windows.UI.Notifications.ToastNotification $xml

    # Set Tag and Group for replacement/threading
    if ($Tag) { $toast.Tag = $Tag }
    if ($Group) { $toast.Group = $Group }

    [Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier($APP_ID).
    Show($toast)
}

# Send multiple notifications
```

```
Show-WindowsNotification -Title "Email Created" -Message "Email to  
john.doe@irs.gov" -Tag "email-001"  
Show-WindowsNotification -Title "Event Created" -Message "Meeting scheduled for  
2PM" -Tag "event-001"  
Show-WindowsNotification -Title "Teams Chat" -Message "Opened chat with Jane" -Tag  
"teams-001"
```

4.3 Notification Throttling Best Practice

[↑ Top](#)

To avoid overwhelming users, implement throttling:

```
$script:lastNotificationTime = $null  
$script:notificationQueue = @()  
  
function Queue-Notification {  
    param([string]$Title, [string]$Message)  
  
    $script:notificationQueue += @{ Title = $Title; Message = $Message; Time =  
Get-Date }  
  
    # Only show one notification per 3 seconds  
    $now = Get-Date  
    if (-not $script:lastNotificationTime -or  
        ($now - $script:lastNotificationTime).TotalSeconds -ge 3) {  
  
        $notification = $script:notificationQueue[0]  
        Show-WindowsNotification -Title $notification.Title -Message  
$notification.Message  
        $script:lastNotificationTime = $now  
        $script:notificationQueue = $script:notificationQueue | Select-Object -  
Skip 1  
    }  
}
```

5. Grouping and Threading

[↑ Top](#)

5.1 Tag and Group System

[↑ Top](#)

Windows supports **Tag** and **Group** properties for notification management:

- **Tag**: Unique identifier within a group. Used to replace or remove specific notifications.
- **Group**: Category identifier. Used to group related notifications together.

5.2 Grouped Notifications Example

[↑ Top](#)

```
# Emails group
Show-WindowsNotification -Title "New Email" -Message "From: Alice" -Tag "email-1"
-Group "emails"
Show-WindowsNotification -Title "New Email" -Message "From: Bob" -Tag "email-2" -
Group "emails"

# Events group
Show-WindowsNotification -Title "Meeting Soon" -Message "Team standup in 10 min" -
Tag "event-1" -Group "events"
Show-WindowsNotification -Title "Meeting Soon" -Message "1:1 with manager in 30
min" -Tag "event-2" -Group "events"
```

Result: Action Center shows these as two separate groups:

- "emails" (2 notifications)
- "events" (2 notifications)

5.3 Header-Based Grouping

[↑ Top](#)

Use `<header>` element for visual grouping with a title:

```
<toast>
  <header id="email-header"
    title="New Emails"
    arguments="group:emails"
    activationType="background"/>
  <visual>
    <binding template="ToastText02">
      <text>Email from Alice</text>
      <text>RE: Project Update</text>
    </binding>
  </visual>
</toast>
```

5.4 Replacing Notifications

[↑ Top](#)

To update an existing notification, use the same **Tag** and **Group**:

```
# Initial notification
Show-WindowsNotification -Title "Download Started" -Message "0% complete" -Tag
```

```
"download-1" -Group "downloads"

# Update (replaces previous)
Show-WindowsNotification -Title "Download Progress" -Message "50% complete" -Tag
"download-1" -Group "downloads"

# Final update (replaces again)
Show-WindowsNotification -Title "Download Complete" -Message "File saved" -Tag
"download-1" -Group "downloads"
```

Only one notification appears in Action Center (the latest one).

6. Silencing Notifications

[↑ Top](#)

6.1 User-Level Controls

[↑ Top](#)

Windows provides several ways for users to control notifications:

6.1.1 Focus Assist (Do Not Disturb)

[↑ Top](#)

Location: Settings → System → Focus Assist

Modes:

- **Off:** All notifications show
- **Priority only:** Only starred apps/contacts notify
- **Alarms only:** Only alarms and timers
- **Automatic rules:** Schedule-based (e.g., during presentations, gaming)

Your App Cannot Override: If Focus Assist blocks your app, notifications are suppressed silently and NOT stored in Action Center.

6.1.2 Per-App Notification Settings

[↑ Top](#)

Location: Settings → System → Notifications & actions → [Your App Name]

User can disable:

- Show notifications in Action Center
- Show notification banners
- Play notification sounds
- Show notification on lock screen

- Number of visible notifications in Action Center (1-20)

Registry Location:

```
HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Notifications\Settings  
\[APP_ID]
```

6.2 Programmatic Silencing

[↑ Top](#)

6.2.1 Silent Notifications (No Banner)

[↑ Top](#)

Show notification in Action Center only (no pop-up):

```
<toast launch="silent">  
  <visual>  
    <binding template="ToastText02">  
      <text>Silent Update</text>  
      <text>Check Action Center for details</text>  
    </binding>  
  </visual>  
  <audio silent="true"/>  
</toast>
```

6.2.2 Suppress Sound

[↑ Top](#)

```
<toast>  
  <visual>...</visual>  
  <audio silent="true"/>  
</toast>
```

6.2.3 Remove Notification Programmatically

[↑ Top](#)

```
function Remove-WindowsNotification {  
  param(  
    [string]$Tag,  
    [string]$Group = ""  
  )  
}
```

```

$APP_ID = "CreateEmailEventAndTeamsChat"
$notifier =
[Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier($APP_ID)
$history = [Windows.UI.Notifications.ToastNotificationManager]::History

if ($Tag) {
    $history.Remove($Tag, $Group, $APP_ID)
}
}

# Remove specific notification
Remove-WindowsNotification -Tag "email-001" -Group "emails"

```

6.2.4 Clear All Notifications for App

[↑ Top](#)

```

function Clear-AllWindowsNotifications {
    $APP_ID = "CreateEmailEventAndTeamsChat"
    $history = [Windows.UI.Notifications.ToastNotificationManager]::History
    $history.Clear($APP_ID)
}

```

7. Notification Persistence and Loss Prevention

[↑ Top](#)

7.1 Do Notifications Get Lost?

[↑ Top](#)

Short Answer: Notifications can be lost in specific scenarios, but Windows has built-in persistence.

7.2 When Notifications Persist

[↑ Top](#)

✅ **Notifications ARE stored** in these cases:

- User is away when notification shows → Stored in Action Center
- User dismisses notification → Stored in Action Center (unless manually cleared)
- System reboots → Notifications survive reboot (Action Center history persists)
- Screen is locked → Notification shown on lock screen (if enabled) + Action Center
- Focus Assist enabled → **Depends on mode** (see below)

7.3 When Notifications Are Lost

[↑ Top](#)

✗ **Notifications ARE NOT stored** in these cases:

- **Focus Assist "Alarms only" mode:** Notification suppressed completely, not saved
- **App notifications disabled in Settings:** Silently dropped
- **Action Center full (20 notification limit):** Oldest notification removed when 21st arrives
- **User clears Action Center:** Manually removed
- **App unregistered/removed:** All associated notifications cleared
- **Notification XML is malformed:** Fails silently, not shown or stored

7.4 Ensuring Notifications Don't Get Lost

[↑ Top](#)

7.4.1 Set Expiration Time

[↑ Top](#)

```
$toast = New-Object Windows.UI.Notifications.ToastNotification $xml
$toast.ExpirationTime = (Get-Date).AddHours(24) # Keep for 24 hours
```

Default expiration: 5 days. Max: 7 days.

7.4.2 Use Scheduled Notifications

[↑ Top](#)

Deliver notification at specific time:

```
$scheduledTime = (Get-Date).AddMinutes(30)
$scheduledToast = New-Object
Windows.UI.Notifications.ScheduledToastNotification($xml, $scheduledTime)
[Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier($APP_ID).
AddToSchedule($scheduledToast)
```

7.4.3 Check Notification History

[↑ Top](#)

Query existing notifications:

```
function Get-WindowsNotificationHistory {
    param([string]$AppId = "CreateEmailEventAndTeamsChat")

    $history = [Windows.UI.Notifications.ToastNotificationManager]::History
    $notifications = $history.GetHistory($AppId)
```

```

foreach ($notification in $notifications) {
    [PSCustomObject]@{
        Tag = $notification.Tag
        Group = $notification.Group
        Content = $notification.Content.GetXml()
    }
}
}

```

7.4.4 Logging and Redundancy

[↑ Top](#)

Maintain your own log of sent notifications:

```

function Show-WindowsNotification-WithLogging {
    param([string]$Title, [string]$Message, [string]$Tag)

    try {
        # Show notification
        Show-WindowsNotification -Title $Title -Message $Message -Tag $Tag

        # Log to file
        $logEntry = [PSCustomObject]@{
            Timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
            Tag = $Tag
            Title = $Title
            Message = $Message
            Status = "Sent"
        }
        $logEntry | Export-Csv -Path ".\NotificationLog.csv" -Append -
        NoTypeInfoInformation
    }
    catch {
        # Log failure
        $logEntry = [PSCustomObject]@{
            Timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
            Tag = $Tag
            Title = $Title
            Message = $Message
            Status = "Failed: $($_.Exception.Message)"
        }
        $logEntry | Export-Csv -Path ".\NotificationLog.csv" -Append -
        NoTypeInfoInformation
    }
}

```

8. Customizing the Toast UI

[↑ Top](#)

8.1 Toast Display Duration

[↑ Top](#)

Default: **7 seconds** on screen, then moves to Action Center

Short Duration (5 seconds):

```
<toast duration="short">...</toast>
```

Long Duration (25 seconds):

```
<toast duration="long">...</toast>
```

8.2 Adding Images

[↑ Top](#)

8.2.1 App Logo Override (Small Icon)

[↑ Top](#)

```
<toast>
  <visual>
    <binding template="ToastGeneric">
      <text>Meeting Reminder</text>
      <text>Team standup in 5 minutes</text>
      <image placement="appLogoOverride"
        hint-crop="circle"
        src="C:\Icons\calendar.png"/>
    </binding>
  </visual>
</toast>
```

Supported Formats: PNG, JPG, GIF (max 200 KB)

8.2.2 Hero Image (Large Banner)

[↑ Top](#)

```
<toast>
  <visual>
    <binding template="ToastGeneric">
      <text>Document Ready</text>
```

```

        <text>Your report has been generated</text>
        <image placement="hero" src="C:\Images\report-preview.jpg"/>
    </binding>
</visual>
</toast>

```

8.2.3 Inline Image

[↑ Top](#)

```

<toast>
    <visual>
        <binding template="ToastGeneric">
            <text>Weather Alert</text>
            <text>Severe thunderstorm warning</text>
            <image src="C:\Weather\radar.png"/>
        </binding>
    </visual>
</toast>

```

8.3 Interactive Buttons

[↑ Top](#)

Add up to 5 buttons:

```

<toast>
    <visual>
        <binding template="ToastGeneric">
            <text>New Email from Alice</text>
            <text>RE: Project Update - Can we meet tomorrow?</text>
        </binding>
    </visual>
    <actions>
        <action content="Reply" arguments="reply:alice@company.com" />
        <action content="Mark Read" arguments="markread:12345" />
        <action content="Delete" arguments="delete:12345" />
    </actions>
</toast>

```

Handling Button Clicks: Requires registering a COM server or using protocol activation (advanced).

8.4 Text Input Fields

[↑ Top](#)

Allow user to type response directly in toast:

```

<toast>
  <visual>
    <binding template="ToastGeneric">
      <text>Quick Reply to Alice</text>
    </binding>
  </visual>
  <actions>
    <input id="textBox" type="text" placeholderContent="Type a reply..." />
    <action content="Send" arguments="send" />
  </actions>
</toast>

```

8.5 Progress Bar

[↑ Top](#)

Show dynamic progress:

```

<toast>
  <visual>
    <binding template="ToastGeneric">
      <text>Downloading File</text>
      <progress value="0.5" valueStringOverride="50%"
status="Downloading..." />
    </binding>
  </visual>
</toast>

```

Update Progress:

```

$data = New-Object Windows.UI.Notifications.NotificationData
$data.Values["progress"] = "0.75"
$data.Values["progressStatus"] = "Almost done..."
$history.Update($data, "download-tag", "downloads", $APP_ID)

```

8.6 Sounds

[↑ Top](#)

8.6.1 System Sounds

[↑ Top](#)

```

<audio src="ms-winsoundevent:Notification.Default"/>
<audio src="ms-winsoundevent:Notification.Mail"/>
<audio src="ms-winsoundevent:Notification.Reminder"/>

```

```
<audio src="ms-winsoundevent:Notification.SMS"/>
<audio src="ms-winsoundevent:Notification.IM"/>
<audio src="ms-winsoundevent:Notification.Looping.Alarm"/>
<audio src="ms-winsoundevent:Notification.Looping.Call"/>
```

8.6.2 Custom Sound

[↑ Top](#)

```
<audio src="file:///C:/Sounds/custom-notification.mp3"/>
```

Supported Formats: MP3, WMA, WAV (max 1 MB, max 30 seconds)

8.6.3 Looping Sound

[↑ Top](#)

```
<audio src="ms-winsoundevent:Notification.Looping.Alarm" loop="true"/>
```

9. Security Model

[↑ Top](#)

9.1 Application ID Registration

[↑ Top](#)

When you call `CreateToastNotifier($APP_ID)`, Windows registers this App ID in:

```
HKEY_CURRENT_USER\Software\Classes\AppUserModelId\[APP_ID]
```

Permissions Required: None (user-level registry access)

Security Implications:

- Any process running as the current user can send notifications under any App ID
- App IDs are **not authenticated** - two different apps can use the same App ID
- No admin rights required

9.2 Notification Content Security

[↑ Top](#)

User Trust Model:

- Notifications display content from the calling application
- Windows does **not validate or sanitize** notification text
- Applications are trusted to show accurate, non-malicious content

Risks:

- **Spoofing:** Malicious app could impersonate another app's App ID
- **Phishing:** Notification could display misleading text/links
- **Spam:** No built-in rate limiting (app must implement)

Mitigations:

- Use unique, namespace-qualified App IDs: `YourCompany.YourApp.Module`
- Validate all user input before showing in notifications
- Include app branding (logo) to prevent confusion
- Don't include sensitive data in notification text (use Action Center detail view)

9.3 Privacy Considerations

[↑ Top](#)**What Windows Logs:**

- Notification content (stored in Action Center database)
- Timestamp of notification
- App ID that sent notification

Location:

```
%LOCALAPPDATA%\Microsoft\Windows\Notifications\wpndatabase.db
```

Retention:

- Up to 20 notifications per app
- Cleared when user clears Action Center
- Can be cleared by IT admin via Group Policy

Cross-Device Sync:

- If user enables "Get notifications from apps and other senders" + "Sync across devices"
- Notifications sync to Microsoft account
- Stored on Microsoft servers temporarily
- **Don't send PII, credentials, or sensitive data in notifications**

9.4 Code Execution Security

[↑ Top](#)

Activation Arguments: When user clicks a toast button, the `arguments` value is passed back to your app:

```
<action content="Open Document" arguments="file://C:\Docs\report.pdf"/>
```

Security Risk: Command injection if arguments are not validated

Safe Pattern:

```
function Handle-NotificationActivation {
    param([string]$Arguments)

    # Validate argument format
    if ($Arguments -match '^action:(\w+):(\d+)$') {
        $action = $Matches[1]
        $id = $Matches[2]

        switch ($action) {
            "reply" { Send-Reply -MessageId $id }
            "delete" { Remove-Item -ItemId $id }
        }
    }
    else {
        Write-Warning "Invalid activation arguments: $Arguments"
    }
}
```

Never use:

```
# DANGEROUS - allows arbitrary code execution
Invoke-Expression $Arguments
```

9.5 Sandboxing

[↑ Top](#)

- Notifications run in the Windows Notification Platform service
- Limited file system access (can load local images)
- No network access from notification XML itself
- Interactive elements (buttons, inputs) require activation contract

10. Multi-Machine Considerations

[↑ Top](#)

10.1 Cross-Device Notifications

[↑ Top](#)

Question: If you show a notification on Machine A, does it appear on Machine B?

Answer: Depends on Windows configuration

10.1.1 Requirements for Sync

[↑ Top](#)

1. User signed in with Microsoft Account (not local account)
2. Settings → System → Notifications → "Get notifications from apps and other senders" = ON
3. Settings → System → Notifications → "Show me notifications from all of my devices" = ON
4. Same app installed on both machines with same App ID

10.1.2 What Syncs

[↑ Top](#)

- Toast notification content (title, message)
- Notification timestamp
- App ID
- Tag and Group

10.1.3 What Doesn't Sync

[↑ Top](#)

- Images (local file paths don't work across machines)
- Progress bars (state not synced)
- Custom sounds (file paths)
- Interactive button state (not synced)

10.2 Best Practices for Multi-Machine Scenarios

[↑ Top](#)

10.2.1 Use HTTP/HTTPS URLs for Images

[↑ Top](#)

Instead of:

```
<image src="C:\Icons\calendar.png"/>
```

Use:

```
<image src="https://yourcdn.com/icons/calendar.png"/>
```

10.2.2 Include Machine Context

[↑ Top](#)

```
$machineName = $env:COMPUTERNAME
Show-WindowsNotification `
  -Title "Document Saved" `
  -Message "Report.docx saved on $machineName"
```

10.2.3 Tag Notifications with Machine ID

[↑ Top](#)

```
$tag = "email-001-$env:COMPUTERNAME"
Show-WindowsNotification -Title "New Email" -Message "From Alice" -Tag $tag
```

This prevents conflicts if same notification sent from different machines.

10.2.4 Server-Side Notification Deduplication

[↑ Top](#)

If using a centralized system:

```
# Server tracks which machine showed notification
$notificationLog = @{
  Id = "email-001"
  ShownOnMachines = @("MACHINE-A", "MACHINE-B")
  Content = "New Email from Alice"
}

# Client queries server before showing
function Show-NotificationIfNotSeen {
  param([string]$NotificationId)

  $machine = $env:COMPUTERNAME
  $status = Get-NotificationStatus -Id $NotificationId -Machine $machine

  if (-not $status.ShownOnThisMachine) {
    Show-WindowsNotification -Title $status.Title -Message $status.Message
    Update-NotificationStatus -Id $NotificationId -Machine $machine -Shown
  }
}

$true
```

10.3 Notification Action Center Sync Behavior

[↑ Top](#)

Important: Action Center sync is **best-effort**:

- Syncs every 15-30 minutes (not real-time)
- Requires internet connection
- May be delayed or dropped if network unreliable
- User can disable sync anytime

Don't rely on sync for critical notifications. Use alternative delivery:

- Email notifications
- Server-side tracking
- Database logging

11. PowerShell Implementation Guide

[↑ Top](#)

11.1 Minimal Working Example

[↑ Top](#)

```
function Show-SimpleToast {
    param(
        [Parameter(Mandatory)][string]$Title,
        [Parameter(Mandatory)][string]$Message
    )

    # Load WinRT types
    [Windows.UI.Notifications.ToastNotificationManager, Windows.UI.Notifications,
    ContentType = WindowsRuntime] | Out-Null
    [Windows.UI.Notifications.ToastNotification, Windows.UI.Notifications,
    ContentType = WindowsRuntime] | Out-Null
    [Windows.Data.Xml.Dom.XmlDocument, Windows.Data.Xml.Dom.XmlDocument,
    ContentType = WindowsRuntime] | Out-Null

    # Create XML
    $template = @"
<toast>
  <visual>
    <binding template="ToastText02">
      <text id="1">$Title</text>
      <text id="2">$Message</text>
    </binding>
  </visual>
</toast>
"@

    # Show notification
    $xml = [Windows.Data.Xml.Dom.XmlDocument]::new()
```

```

$xml.LoadXml($template)
$toast = [Windows.UI.Notifications.ToastNotification]::new($xml)

[Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier("MyApp").
Show($toast)
}

# Usage
Show-SimpleToast -Title "Test" -Message "Hello World"

```

11.2 Production-Ready Implementation

[↑ Top](#)

```

function Show-WindowsNotification {
    <#
    .SYNOPSIS
        Shows a Windows toast notification with advanced options.

    .PARAMETER Title
        Notification title (first line, bold text)

    .PARAMETER Message
        Notification message (second line, regular text)

    .PARAMETER AppId
        Application identifier for grouping. Default:
        "CreateEmailEventAndTeamsChat"

    .PARAMETER Tag
        Unique tag for this notification (enables replacement/removal)

    .PARAMETER Group
        Group identifier for related notifications

    .PARAMETER ImagePath
        Path to image file for app logo override (circle icon)

    .PARAMETER Sound
        Sound to play: Default, Mail, Reminder, SMS, IM, Silent

    .PARAMETER Duration
        How long to display: Short (5s), Long (25s), Default (7s)
    #>

    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$Title,

        [Parameter(Mandatory)]

```

```

    [string]$Message,

    [string]$AppId = "CreateEmailEventAndTeamsChat",

    [string]$Tag = $null,

    [string]$Group = $null,

    [ValidateScript({Test-Path $_ -PathType Leaf})]
    [string]$ImagePath = $null,

    [ValidateSet('Default', 'Mail', 'Reminder', 'SMS', 'IM', 'Silent')]
    [string]$Sound = 'Default',

    [ValidateSet('Short', 'Long', 'Default')]
    [string]$Duration = 'Default'
)

try {
    # Load WinRT types
    [Windows.UI.Notifications.ToastNotificationManager,
Windows.UI.Notifications, ContentType = WindowsRuntime] | Out-Null
    [Windows.UI.Notifications.ToastNotification, Windows.UI.Notifications,
ContentType = WindowsRuntime] | Out-Null
    [Windows.Data.Xml.Dom.XmlDocument, Windows.Data.Xml.Dom.XmlDocument,
ContentType = WindowsRuntime] | Out-Null

    # Escape XML special characters
    $TitleEscaped = [System.Security.SecurityElement]::Escape($Title)
    $MessageEscaped = [System.Security.SecurityElement]::Escape($Message)

    # Build toast duration attribute
    $toastDuration = if ($Duration -ne 'Default') { "
duration=`"$($Duration.ToLower())`"" } else { "" }

    # Build image XML if provided
    $imageXml = ""
    if ($ImagePath) {
        $ImagePathEscaped =
[System.Security.SecurityElement]::Escape($ImagePath)
        $imageXml = "<image placement=`"appLogoOverride`" hint-crop=`"circle`"
src=`"$ImagePathEscaped`"/>"
    }

    # Build audio XML
    $soundSrc = switch ($Sound) {
        'Default' { "ms-winsoundevent:Notification.Default" }
        'Mail' { "ms-winsoundevent:Notification.Mail" }
        'Reminder' { "ms-winsoundevent:Notification.Reminder" }
        'SMS' { "ms-winsoundevent:Notification.SMS" }
        'IM' { "ms-winsoundevent:Notification.IM" }
        'Silent' { $null }
    }
}

```

```

        $audioXml = if ($Sound -eq 'Silent') {
            '<audio silent="true"/>'
        } elseif ($soundSrc) {
            "<audio src=`"$soundSrc`"/>"
        } else {
            ""
        }

        # Build XML template
        $template = @"
<toast$toastDuration>
    <visual>
        <binding template="ToastGeneric">
            <text>$TitleEscaped</text>
            <text>$MessageEscaped</text>
            $imageXml
        </binding>
    </visual>
    $audioXml
</toast>
"@

        # Create and show notification
        $xml = [Windows.Data.Xml.Dom.XmlDocument]::new()
        $xml.LoadXml($template)
        $toast = [Windows.UI.Notifications.ToastNotification]::new($xml)

        # Set Tag and Group
        if ($Tag) { $toast.Tag = $Tag }
        if ($Group) { $toast.Group = $Group }

        [Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier($AppId).Show($toast)

        Write-Verbose "Notification shown: Title='$Title', Tag='$Tag',
Group='$Group'"
        return $true
    }
    catch {
        Write-Error "Failed to show notification: $($_.Exception.Message)"
        return $false
    }
}

```

11.3 Helper Functions

[↑ Top](#)

```

function Remove-WindowsNotification {
    param(

```

```

        [Parameter(Mandatory)][string]$Tag,
        [string]$Group = "",
        [string]$AppId = "CreateEmailEventAndTeamsChat"
    )

    $history = [Windows.UI.Notifications.ToastNotificationManager]::History
    $history.Remove($Tag, $Group, $AppId)
}

function Clear-AllWindowsNotifications {
    param([string]$AppId = "CreateEmailEventAndTeamsChat")

    $history = [Windows.UI.Notifications.ToastNotificationManager]::History
    $history.Clear($AppId)
}

function Get-AllWindowsNotificationHistory {
    param([string]$AppId = "CreateEmailEventAndTeamsChat")

    $history = [Windows.UI.Notifications.ToastNotificationManager]::History
    $notifications = $history.GetHistory($AppId)

    foreach ($notification in $notifications) {
        [PSCustomObject]@{
            Tag = $notification.Tag
            Group = $notification.Group
            ExpirationTime = $notification.ExpirationTime
            Content = $notification.Content.GetXml()
        }
    }
}

```

12. Advanced Features

[↑ Top](#)

12.1 Scheduled Notifications

[↑ Top](#)

Deliver notification at future time:

```

function Schedule-AllWindowsNotification {
    param(
        [string]$Title,
        [string]$Message,
        [datetime]$DeliveryTime,
        [string]$AppId = "CreateEmailEventAndTeamsChat"
    )
}

```

```

[Windows.UI.Notifications.ToastNotificationManager, Windows.UI.Notifications,
ContentType = WindowsRuntime] | Out-Null
[Windows.UI.Notifications.ScheduledToastNotification,
Windows.UI.Notifications, ContentType = WindowsRuntime] | Out-Null
[Windows.Data.Xml.Dom.XmlDocument, Windows.Data.Xml.Dom.XmlDocument,
ContentType = WindowsRuntime] | Out-Null

$template = @"
<toast>
  <visual>
    <binding template="ToastText02">
      <text id="1">$Title</text>
      <text id="2">$Message</text>
    </binding>
  </visual>
</toast>
"@

$xml = [Windows.Data.Xml.Dom.XmlDocument]::new()
$xml.LoadXml($template)

$scheduledToast =
[Windows.UI.Notifications.ScheduledToastNotification]::new($xml, $DeliveryTime)

[Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier($AppId).Add
ddToSchedule($scheduledToast)
}

# Schedule notification for 30 minutes from now
Schedule-WindowsNotification -Title "Meeting Reminder" -Message "Team sync in 5
minutes" -DeliveryTime (Get-Date).AddMinutes(30)

```

12.2 Notification Update (Progress Bar)

[↑ Top](#)

```

function Show-ProgressNotification {
    param(
        [string]$Title,
        [double]$Progress, # 0.0 to 1.0
        [string]$Status,
        [string]$Tag = "progress",
        [string]$AppId = "CreateEmailEventAndTeamsChat"
    )

    [Windows.UI.Notifications.ToastNotificationManager, Windows.UI.Notifications,
    ContentType = WindowsRuntime] | Out-Null
    [Windows.UI.Notifications.NotificationData, Windows.UI.Notifications,
    ContentType = WindowsRuntime] | Out-Null

    $progressValue = [Math]::Round($Progress, 2)

```

```

    $progressPercent = [Math]::Round($Progress * 100)

    $template = @"
<toast>
  <visual>
    <binding template="ToastGeneric">
      <text>$Title</text>
      <progress value="$progressValue"
valueStringOverride="$progressPercent%" status="$Status" />
    </binding>
  </visual>
</toast>
"@

# Show initial notification
$xml = [Windows.Data.Xml.Dom.XmlDocument]::new()
$xml.LoadXml($template)
$toast = [Windows.UI.Notifications.ToastNotification]::new($xml)
$toast.Tag = $Tag

[Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier($AppId).Show($toast)
}

# Usage: Show progress bar
Show-ProgressNotification -Title "Downloading File" -Progress 0.0 -Status "Starting..."
Start-Sleep -Seconds 2
Show-ProgressNotification -Title "Downloading File" -Progress 0.5 -Status "50% complete"
Start-Sleep -Seconds 2
Show-ProgressNotification -Title "Downloading File" -Progress 1.0 -Status "Complete!"

```

12.3 Notification with Buttons (Advanced)

[↑ Top](#)

```

function Show-InteractiveNotification {
    param(
        [string]$Title,
        [string]$Message,
        [string]$AppId = "CreateEmailEventAndTeamsChat"
    )

    $template = @"
<toast launch="app-action">
  <visual>
    <binding template="ToastGeneric">
      <text>$Title</text>
      <text>$Message</text>

```

```
</binding>
</visual>
<actions>
  <action content="View" arguments="action=view" />
  <action content="Dismiss" arguments="action=dismiss" />
</actions>
</toast>
"@

$xml = [Windows.Data.Xml.Dom.XmlDocument]::new()
$xml.LoadXml($template)
$toast = [Windows.UI.Notifications.ToastNotification]::new($xml)

# Note: Handling button clicks requires COM activation or protocol handler
# See: https://learn.microsoft.com/en-us/windows/apps/design/shell/tiles-and-notifications/send-local-toast

[Windows.UI.Notifications.ToastNotificationManager]::CreateToastNotifier($AppId).Show($toast)
}
```

Note: Interactive notifications require additional setup:

- Register COM server for activation
- Or use protocol activation (e.g., `myapp://action=view`)

13. Summary and Feature Matrix

[↑ Top](#)

Summary Table

Feature	Capability	PowerShell Support
Basic toast (title + message)	✓ Yes	✓ Full support
Images (logo, hero, inline)	✓ Yes	✓ Full support
Custom sounds	✓ Yes	✓ Full support
Silent notifications	✓ Yes	✓ Full support
Duration control (short/long)	✓ Yes	✓ Full support
Tag and Group	✓ Yes	✓ Full support
Replace/Remove notifications	✓ Yes	✓ Full support
Progress bar	✓ Yes	✓ Full support
Scheduled delivery	✓ Yes	✓ Full support

Feature	Capability	PowerShell Support
Action Center history	✓ Yes	✓ Full support
Interactive buttons	⚠ Limited	⚠ Requires COM/protocol handler
Text input	⚠ Limited	⚠ Requires COM/protocol handler
Button click handling	⚠ Limited	⚠ Requires COM/protocol handler
Cross-device sync	⚠ User-controlled	N/A (Windows feature)
Security/Authentication	✗ No	N/A (not supported by Windows)

References

- [Windows Toast Notifications Official Docs](#)
- [Toast Content Schema](#)
- [Windows.UI.Notifications Namespace](#)
- [Notification Activation \(Interactive\)](#)

14. Glossary

[↑ Top](#)

Action Center: Windows notification history panel accessible via Win+A keyboard shortcut. Stores up to 20 notifications per application.

Adaptive Template: Modern toast notification layout (ToastGeneric) that supports rich content like images, progress bars, and buttons.

App ID (Application ID): Unique identifier string used to register and group notifications from the same application.

App Logo Override: Small circular icon image that replaces the default app icon in a toast notification.

Audio Element: XML tag in toast template that specifies notification sound (system sound or custom file).

BurntToast: Third-party PowerShell module for Windows notifications (not used in this implementation).

COM (Component Object Model): Microsoft's binary interface standard for inter-process communication.

ContentType = WindowsRuntime: PowerShell syntax for loading WinRT (Windows Runtime) assemblies into the session.

Cross-Device Sync: Optional Windows feature that syncs notifications across devices signed into the same Microsoft Account.

Duration: How long a toast notification remains visible on screen (Short=5s, Default=7s, Long=25s).

Expiration Time: When a notification should be removed from Action Center (default 5 days, max 7 days).

Focus Assist: Windows Do Not Disturb mode that suppresses notifications based on rules (Off, Priority only, Alarms only).

GAL (Global Address List): The organization-wide Outlook address book (not directly related to notifications).

Group: Category identifier for related notifications enabling threading in Action Center.

Header: XML element that provides a title for grouped notifications in Action Center.

Hero Image: Large banner image displayed at the top of a toast notification.

Inline Image: Medium-sized image displayed within the notification body content.

Interactive Notification: Toast with buttons or input fields that users can click or type into.

MAPI (Messaging Application Programming Interface): Microsoft's email client protocol (not used for toast notifications).

Microsoft Account: User account format (email@domain.com) that enables cross-device sync.

NDR (Non-Delivery Report): Email bounce-back notification (not related to toast notifications).

Notification Data: Object containing key-value pairs for updating dynamic notification content (e.g., progress bars).

Notification History: Collection of notifications stored in Action Center, queryable via `ToastNotificationManager.History`.

Per-App Settings: Windows notification settings specific to one application (sound, banner, Action Center visibility).

Priority Only: Focus Assist mode that only shows notifications from starred apps or contacts.

Progress Bar: Visual indicator in toast notification showing completion percentage with status text.

Protocol Activation: Method for handling interactive button clicks using custom URL schemes (e.g., `myapp://action`).

PWA (Progressive Web App): Web application that runs like a native app (New Outlook is a PWA).

Registry: Windows hierarchical database storing system and app settings, including notification App IDs.

Scheduled Notification: Toast notification set to appear at a future time rather than immediately.

Silent Notification: Toast that appears in Action Center without displaying a banner or playing sound.

SQLite Database: File-based database format used by Windows to store Action Center notifications (`wpndatabase.db`).

System Sound: Built-in Windows notification sound (Default, Mail, Reminder, SMS, IM, Alarm, Call).

Tag: Unique identifier for a notification within a Group, used for replacement or removal.

Toast: Pop-up notification that appears in the lower-right corner of the screen.

Toast Notification Manager: WinRT class that creates, shows, schedules, and manages toast notifications.

ToastGeneric: Modern adaptive template type supporting rich content and layouts.

ToastText02: Legacy template type with two text lines (title and message).

WinRT (Windows Runtime): Microsoft's API for Windows Store apps and modern Windows features like notifications.

wpndatabase.db: SQLite database file storing Action Center notification history.

XAML (Extensible Application Markup Language): XML-based UI definition language for Windows applications.

XML Template: String defining notification content structure following the Toast Content Schema.

[Windows.Data.Xml.Dom.XmlDocument]: WinRT class for creating and manipulating XML documents.

[Windows.UI.Notifications.ToastNotification]: WinRT class representing a single toast notification.

[Windows.UI.Notifications.ToastNotificationManager]: WinRT class for showing and managing toast notifications.

End of Document

[↑ Top](#)

Document Version: 2.0

Last Updated: 2026-09-19

For: CreateEmailEventAndTeamsChat.Prototype.ps1 Notifications Feature